Proof-Driven Multi-Module Python via Agentic TLA$^+$ Co-Synthesis

Harry Wang Eric Shang Nan Chen

May 2026

Abstract

LLM code generators score well on single-function benchmarks but collapse on multi-function compositional tasks: a recent benchmark, DafnyComp [1], measures a drop from ≈ 58% end-to-end verification on single-function problems to 3.69% on compositional ones. We attack that gap from the TLA$^+$ side. We build an agentic pipeline that, from a natural-language requirement, (i) plans a parent-plus-children decomposition, (ii) co-synthesises an abstract spec and an implementation PlusCal module for each child, (iii) discharges four classes of TLC proof obligations — initiation, consecution, property implication, and a cross-module refinement obligation in the Hillel-Wayne ADT style [7] — with structured counterexample-driven repair, (iv) refines the verified bundle into an `icontract`-decorated [6] Python package, and (v) runs an advisory trace-conformance gate that replays one seeded Python execution against each child's abstract spec via TLC, using the encoding of Cirstea et al. [3]. The pipeline ships with two legacy single-module baselines and six compositional benchmarks covering producer/ consumer queues, write-through caches, ledger audits, two-phase commit, leader election, and a reader-writer lock. The offline test suite (24 files, 182 cases) is green; live end-to-end evaluation against the benchmark suite is in progress.

1 Introduction

LLM-driven program synthesis has converged on a write-first, verify-second loop: emit Python (or Dafny, or Rust), then run a test or a verifier and ask the model to repair. This loop works on isolated functions, but the recent DafnyComp benchmark [1] shows that on compositional, multi-function tasks, state-of-the-art LLMs hit only 3.69% verified outputs — a ≈ 92-point gap versus the single-function case. The bottleneck is not syntax: DafnyComp reports 95.67% syntactic correctness even on the compositional split. The bottleneck is reasoning about how multiple modules, each with its own invariants, fit together into a globally correct system.

TLA$^+$ [4] is an attractive verification target for that gap, for three reasons. First, TLA$^+$'s refinement composition pattern [7] lets us discharge whole-system correctness as a conjunction of per-child refinements, each verifiable by the same model-checker (TLC [8]). Second, PlusCal [5] gives LLMs an imperative algorithmic surface to write, which then translates mechanically to TLA$^+$'s state-transition style. Third, TLC's counterexamples are structured state sequences that an LLM can read and revise — the repair signal is concrete rather than a binary pass/fail.

This paper describes a pipeline that exploits all three properties. Our contributions are:

- An LLM-driven *decomposition* stage that emits a typed `DecompositionPlan` (parent + up to four child modules, each with an abstract-interface sketch).

- A *bundled co-synthesis* stage that emits, for each child, both an abstract spec module and an implementation module with an explicit abstraction map back to parent variables.
- A *four-obligation verifier* that runs three per-implementation TLC checks (initiation, consecution, property implication) plus a cross-module refinement check, with structured counterexamples piped back to the synthesis agent via Anthropic's `tool_result` channel.
- A *Python refinement* stage that emits one `icontract`-decorated [6] class per implementation module plus a parent application that composes them.
- A *trace-conformance gate* that subprocesses the refined Python package, captures every action and abstract-state observation as JSONL, and replays it against each child's abstract spec via TLC, using the encoding of Cirstea et al. [3].
- A six-program compositional benchmark suite covering classical concurrency/consistency problems: producer–consumer, write-through cache, ledger audit, two-phase commit, leader election, and a reader-writer lock.

The remainder of the paper is organised as follows. Section 2 reviews the necessary TLA$^+$ and DafnyComp context. Section 3 describes the five pipeline stages and the four proof obligations in detail. Section 4 summarises the implementation. Section 5 describes the benchmark suite and the offline test status, and reserves a results table for the in-progress live sweep. Section 6 states limitations and the next planned hardening pass. Section 7 concludes.

2 Background and Related Work

TLA$^+$, PlusCal, TLC. TLA$^+$ [4] specifies systems as state machines using a temporal-logic formula $\textsc{Init} \wedge \Box[\textsc{Next}]_{vars}$. PlusCal [5] is an algorithmic notation that compiles to TLA$^+$ (via `pcal.trans`). TLC [8] is the explicit-state model checker for TLA$^+$ that we drive as a subprocess.

Refinement composition. Our composition pattern follows Hillel Wayne's TLA$^+$-for-ADTs note [7]: a parent module declares variables and defines its own Init/Next, and each child is brought in as a named `INSTANCE` of its *abstract* module via `WITH` substitutions that bind the abstract variables to expressions over the parent's variables (the abstraction map). The whole-system refinement obligation is then $\bigwedge_i$ `Abs`$_i$`!Spec`. This pattern composes at the TLA$^+$ layer only — PlusCal itself does not compose — so our pipeline glues children together after `pcal.trans` has run on each implementation module.

DafnyComp. DafnyComp [1] is the September 2025 benchmark that motivates this work. It measures LLM-driven verified code generation on both single-function and compositional multi-function tasks; the collapse from $\approx 58\%$ to 3.69% verification rate on the compositional split is the headline gap our pipeline targets, from the TLA$^+$ side rather than the Dafny side.

Trace conformance. Cirstea et al. [3] (§3.2) encode a runtime trace as a literal sequence in TLA$^+$ and let TLC verify that the trace is admitted by a spec. We adopt their encoding verbatim for our Week-3 trace gate, with the modification that conformance is inferred from the depth TLC reports against the trace length, since TLC 2.19 does not implement a `POSTCONDITION` primitive.

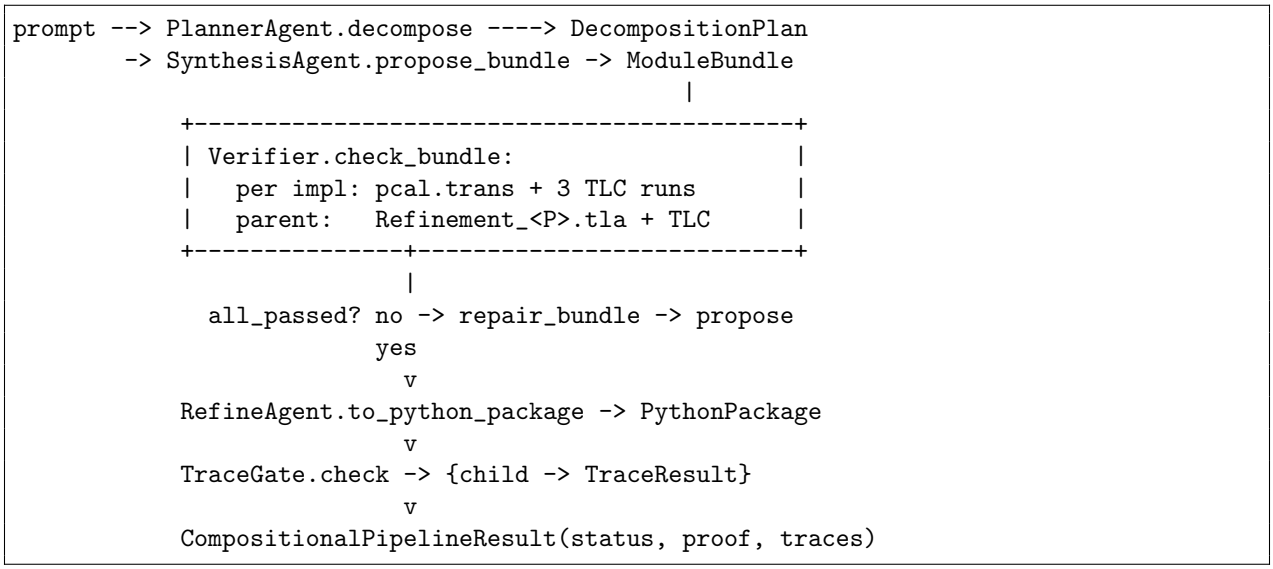

Figure 1: The compositional pipeline. Arrows are typed data flow; the repair loop is the only control-flow back-edge.

Out of scope. We do not use TLAPS for unbounded inductive proofs; all our proofs are by finite model-checking with explicit small constant domains. We do not use Nagini or Viper for Python verification; our Python guarantee is the conjunction of (i) static refinement and (ii) runtime `icontract` assertions plus the trace-conformance gate. We do not attempt liveness; all four obligations are safety obligations.

3 Approach

The pipeline has five stages (Figure 1). All five are wired through `run_compositional_pipeline` in `src/main.py`; the legacy single-module path (`run_pipeline`) is preserved for the two non-compositional baselines.

3.1 Decomposition

`PlannerAgent.decompose` (`src/agents/planner_agent.py`) calls Claude Opus 4.7 with a single function-calling tool, `propose_decomposition`, that emits a `DecompositionPlan` consisting of a parent name plus 1–4 child modules. Each child carries an `AbstractInterface` (state variables, action names, invariant sketch). Decomposition is retried up to two times on Pydantic validation failure; exhaustion raises `DecompositionError` and the pipeline returns `status="planner_failed"`.

3.2 Bundled Co-Synthesis

`SynthesisAgent.propose_bundle` (`src/agents/synth_agent.py`) takes the plan and asks the LLM, via `propose_module_bundle`, for a `ModuleBundle`: a parent TLA$^+$ module plus, for each child, an *abstract* module and an *implementation* module that carries both PlusCal source and an `abstraction_map` from abstract variables to expressions over the parent's variables. The bundle schema (Pydantic classes in `src/models/bundle.py`) enforces role consistency: only implementations may carry PlusCal or an abstraction map; only one parent is allowed.

Obligation	Formula	TLC encoding (file)
Initiation	$\textsc{Init} \Rightarrow \textsc{Inv}$	`SPECIFICATION Spec, INVARIANT Inv` on `<Name>_Impl.tla` (`tlc_config.py:write_init_cfg`)
Consecution	$\textsc{Inv} \wedge \textsc{Next} \Rightarrow \textsc{Inv}'$	Aux `Consec_<Name>_Impl.tla` with `IInit == Inv`, `ISpec == IInit /\ [][Next]_vars` (`tlc_config.py:write_consec_module`)
Property impl.	$\textsc{Inv} \Rightarrow \textsc{Property}$	Aux `Prop_<Name>_Impl.tla` with `PInit == Inv`, unchanged transition (`tlc_config.py:write_property_module`)
Refinement	each `Impl` refines its `Abs`	Aux `Refinement_<Parent>.tla`; see Listing 1 (`formal/refinement.py:generate_refinement_module`)

Table 1: The four proof obligations and their TLC encodings.

Listing 1: Generated `Refinement_Parent.tla` (simplified). The `const_clauses` and `var_clauses` come from the abstract module's `CONSTANTS` declaration and the implementation's `abstraction_map`, respectively.

```
---- MODULE Refinement_Parent ----
EXTENDS Parent
Abs_BackingStore == INSTANCE BackingStore_Abs WITH
    Keys <- Keys, Vals <- Vals,         (* CONSTANTS from abs *)
    storePresent <- store_p,            (* abstraction_map    *)
    storeVal     <- store_v
Abs_Cache == INSTANCE Cache_Abs WITH ...
RefinementSpec == /\ Abs_BackingStore!Spec
                  /\ Abs_Cache!Spec
====
```

3.3 Four proof obligations

Given a bundle the verifier (`src/agents/verifier.py`) discharges four classes of obligation, summarised in Table 1.

The first three obligations are per-implementation and reuse the single-module encoding from the May 6 progress report. The fourth is the compositional obligation: for the bundle as a whole, each child must refine its abstract spec under the parent's substitution. We render this as the Hillel-Wayne ADT pattern [7], shown in Listing 1: the auxiliary module *extends* the parent, defines one named `INSTANCE` of the abstract module per child with `WITH` substitutions for both the abstract variables and any abstract constants, and asserts the conjunction of all $\texttt{Abs}_i\texttt{!Spec}$ as a temporal property.

TLC is then invoked on `Refinement_<Parent>.tla` with `SPECIFICATION Spec` (the parent's composed behaviour) and `PROPERTY RefinementSpec`, which fails as soon as any child's abstract behaviour is not admitted by its implementation under the abstraction map.

3.4 Counterexample-driven repair

TLC's stdout is parsed by `src/formal/counterexample_parser.py` into an `ObligationResult` (one per failed check) carrying the violated predicate and the offending state sequence. The synthesis agent's `_serialise_bundle_failure` flattens these per-implementation and refinement failures into a single `tool_result` block that the LLM sees on its next turn. The system prompt is cached

Listing 2: Generated `Trace_<Name>.tla`, replaying one execution against the child's abstract spec.

```
---- MODULE Trace_Queue ----
EXTENDS Queue_Abs, Sequences, Naturals, TLC
Trace == << [event |-> "Enqueue", q |-> << >>],
            [event |-> "Enqueue", q |-> << 1 >>], ... >>
VARIABLE l
IsEvent(e) ==
    /\ l \in 1..(Len(Trace) - 1)
    /\ Trace[l + 1].event = e
    /\ q' = Trace[l + 1].q
    /\ l' = l + 1
TraceInit == l = 1 /\ q = Trace[1].q
TraceNext == \/ IsEvent("Enqueue") /\ Enqueue
             \/ IsEvent("Dequeue") /\ Dequeue
TraceSpec == TraceInit /\ [][TraceNext]_<<l, q>>
====
```

via Anthropic ephemeral `cache_control` so that repair iterations re-use the same prompt without paying the input-token cost again.

3.5 Refinement to `icontract` Python

On a verified bundle, `RefineAgent.to_python_package` (`src/agents/refine_agent.py`) makes one LLM call per implementation to emit a Python module — one class per implementation, with `@icontract.invariant` for each conjunct of the verified Inv, and `@require`/`@ensure` for each method — then one more call to emit a parent `app.py` that composes the children. Each mutating method calls a tiny `log_action(name, state)` shim that appends one JSONL line of (action name, abstract-variable snapshot) to a trace file at runtime; this is the substrate for the trace gate.

3.6 Trace conformance (advisory)

`TraceGate.check` (`src/agents/trace_gate.py`) subprocesses the emitted package once with a seeded RNG and a step budget (default 50), collects the `trace.jsonl` that the `log_action` shim produced, partitions entries by class prefix, and for each child renders a `Trace_<Name>.tla` module (Listing 2) per Cirstea et al. [3].

TLC is then run with `SPECIFICATION TraceSpec, INVARIANT Inv`. Since TLC 2.19 has no `POSTCONDITION`, we classify the outcome from TLC's reported depth: depth equal to the trace length with no error is `conforms`; smaller depth with no error is `diverged at step` $K+1$; an invariant violation is `invariant_violated`; the rest is `tlc_timeout` or `tlc_error`. Crucially, trace failures are *advisory only*: `CompositionalPipelineResult.status` is set by the four static obligations, and trace results are reported alongside but do not flip a verified bundle to unverified. The rationale is that the static obligations already prove refinement over *all* behaviours; the trace gate is runtime evidence that the actual Python execution stays inside that refinement.

4 Implementation

The repository layout is shown in Figure 2. The LLM backbone is Claude Opus 4.7 [2] with a three-tier overload fallback: `claude-opus-4-7` $\xrightarrow{529}$ `claude-opus-4-6` $\xrightarrow{529}$ a tertiary OpenAI

```
src/
  agents/   planner_agent, synth_agent, verifier,
            refine_agent, trace_gate
  formal/   pcal_translator, tla_runner, tlc_config,
            counterexample_parser, refinement, trace_renderer,
            trace_postcondition
  llm/      anthropic_client, openai_adapter, tools, prompts
  models/   task, decomposition, synthesis, bundle, proof
  main.py   run_pipeline + run_compositional_pipeline
  cli.py    Typer CLI (--compositional/--legacy)
examples/   8 benchmarks (2 legacy + 6 compositional)
scripts/    run_benchmarks.py (CSV + Markdown sweep)
tests/      24 test files, 182 cases (offline + TLC-gated + e2e)
```

Figure 2: Repository layout.

adapter that translates between Anthropic-shape and OpenAI-shape `tool_use`. The verifier engine is TLC 2.19, driven as a subprocess with `-config <obligation>.cfg -workers auto -deadlock`.

The data plane is a single `CompositionalPipelineResult` (`src/models/task.py`) carrying `status`, the `DecompositionPlan`, the verified `ModuleBundle`, the `CompositionalProofBundle` (one `ProofBundle` per child plus the refinement `ObligationResult`), the emitted `PythonPackage`, the `traces` dict (one `TraceResult` per child), and a `trace_skipped_reason` when the gate was not run (e.g. because the bundle was unverified).

5 Evaluation

5.1 Offline test suite

The repository ships 24 `pytest` files exercising the pipeline at three boundary levels: pure unit tests of the data models, the counterexample parser, the trace renderer, and the refinement-module generator; mocked-LLM tests of the planner, synthesiser, and refiner that replay captured tool-call traces; and TLC-gated integration tests that verify the trace gate end-to-end against a real `tla2tools.jar` and exercise the legacy bounded-counter fixtures. The offline suite currently runs at **182/182** passing without an Anthropic API key or `tla2tools.jar`, with external dependencies mocked at the boundary.

5.2 Benchmark suite

The benchmark suite is two legacy single-module baselines plus six compositional programs (Table 2). Each compositional benchmark ships a `prompt.txt` and a reference `expected_modules.yaml` used only for evaluation diff — the expected decomposition is *not* forced on the planner.

5.3 Live sweep (in progress)

The live benchmark sweep is driven by `scripts/run_benchmarks.py`, which writes one CSV row per benchmark with the columns `slug, status, iters, n_impls, ref_status, per_module_pass, per_module_fail, trace_conform, trace_diverged, trace_inv_violated, trace_other, pipeline_seconds, total_tlc_seconds, error_note`. Table 3 reports the most recent sweep; the target is at least four of six compositional benchmarks reaching `status="verified"`.

Slug	Modules	Safety property
bounded_counter	1 (single)	bounded in 0..10
bank_transfer	1 (single)	sum preserved
producer_consumer	Queue (abs+impl), Producer, Consumer	no item lost; bounded queue
cache_store	Cache (abs+impl), Store	read-after-write consistency
bank_audit	Account, Ledger, Transfer	sum preserved; ledger $\Leftrightarrow$ balance
two_phase_commit	ResourceManager, Coordinator	agreement; no commit$\rightarrow$abort
leader_log	Node, Election	≤ 1 leader/term; log monotone
rw_lock	RwLock, Client	mutex on write; reader_count

Table 2: Benchmark suite. The six compositional benchmarks were hand-authored to cover producer/consumer queues, caches/stores, ledgers, distributed commit, leader election, and locking.

Slug	Status	Iters	Init/Consec/Prop	Refinement	Traces	TLC s
bounded_counter	—	—	—	—	—	—
bank_transfer	—	—	—	—	—	—
producer_consumer	—	—	—	—	—	—
cache_store	—	—	—	—	—	—
bank_audit	—	—	—	—	—	—
two_phase_commit	—	—	—	—	—	—
leader_log	—	—	—	—	—	—
rw_lock	—	—	—	—	—	—

Table 3: Live benchmark sweep results (to be filled in after the next end-to-end run).

5.4 Qualitative observations from early live runs

The first end-to-end run with live LLM tokens surfaced two systematic failure modes that were not visible from offline tests. First, the LLM repeatedly emitted PlusCal blocks whose labels collided with PlusCal reserved names (`Done`, `Error`, `Lbl_N`), causing `pcal.trans` to reject the module before TLC was ever invoked. Second, the refinement aux module generator initially omitted `CONSTANTS` substitutions in the `INSTANCE` clauses, causing SANY to fail to link the parent's variables to the abstract modules and report cascading "unknown operator" errors. Both traps were observable from inside TLC's stdout and parsed by our counterexample channel, but the LLM regenerated the same structural error across all three repair iterations: the repair signal was informative but the model lacked the mechanical rule. The fix landed in two places: (i) the refinement generator now parses abs-module `CONSTANTS` and emits the matching `WITH` bindings, and (ii) the bundle synthesis system prompt explicitly forbids the reserved PlusCal labels. These structural rules turned out to be load-bearing for the rest of the suite, which justifies the planned hardening pass below.

6 Limitations and Future Work

Limitations. The pipeline targets finite-domain inductive checking with TLC; an unbounded inductive proof would need TLAPS, which is out of scope. Inv must be expressible as an initial-state predicate, so each variable must appear under explicit set membership. The trace gate is per-child, against the child's abstract spec; parent-level trace replay would require richer LLM coupling of Python state to parent TLA$^+$ variables. Parametric abstract actions are degraded: when an abstract operator takes arguments (e.g. `Enqueue(item)`), the trace gate currently cannot reliably invent the

parameter domain and keeps only the per-state Inv check, dropping the action-relation conjunct. Finally, the trace gate assumes the emitted Python is deterministic: `random` is seeded at module import and the subprocess runs under `PYTHONHASHSEED=0`; threading and unseeded randomness are out of scope.

Future work. Four follow-on items are queued. (i) *Status split.* The single `status` field on `CompositionalPipelineResult` splits into a triple of `formal_status`, `python_status`, and `overall_status`, so the CLI's exit code reflects runtime crashes in the emitted package, not just static verification. (ii) *Parametric-action fix.* `log_action` grows a `params` kwarg; the renderer reads it and emits the action-relation conjunct, closing the parametric-action degradation. (iii) *CrossHair gate.* `crosshair-tool` is added as an optional gate behind a `--crosshair` flag, providing an additional symbolic test of the emitted Python contracts. (iv) *Enriched benchmark report.* `run_benchmarks.py` is extended with per-stage pass rates (TLC-runnable, invariant, refinement, trace-conformance) plus action coverage and repair-iteration count, so the headline DafnyComp-style comparison can be reported directly.

7 Conclusion

We have described an agentic TLA$^+$ pipeline that targets the DafnyComp [1] compositional-verification gap from the TLA$^+$ side. The pipeline plans a parent-plus-children decomposition, co-synthesises abstract and implementation modules for each child, discharges three per-implementation TLC obligations plus a cross-module refinement obligation in the Hillel-Wayne ADT style [7], emits an `icontract`-decorated Python package, and runs an advisory trace-conformance gate [3] against each child's abstract spec. The four-obligation static proof is the load-bearing contribution; the trace gate is a runtime witness that the emitted Python stays inside the proved refinement. The offline test suite is green at 182/182, the benchmark suite is built and being swept, and the next hardening pass is already scoped against the issues observed in early live runs.

References

[1] Anonymous. DafnyComp: A compositional benchmark for LLM-driven verified code generation. arXiv preprint arXiv:2509.23061, September 2025. Reports a collapse from $\approx 58\%$ verification on single-function tasks to 3.69% on multi-function compositional tasks for state-of-the-art LLMs.

[2] Anthropic. Claude Opus 4.7 and the Anthropic Python SDK. https://docs.anthropic.com/, 2026.

[3] Andra Cîrstea et al. Trace conformance checking in TLA+. arXiv preprint arXiv:2404.16075, 2024. §3.2 describes the per-trace replay encoding adopted by our Week-3 trace-conformance gate.

[4] Leslie Lamport. *Specifying Systems: The TLA+ Language and Tools for Hardware and Software Engineers*. Addison-Wesley, 2002.

[5] Leslie Lamport. The +CAL algorithm language. In *Theoretical Aspects of Computing (ICTAC)*, volume 5684 of *LNCS*, pages 36–60. Springer, 2009.

[6] Parquery AG. `icontract`: Design-by-contract decorators for Python. Software, https://github.com/Parquery/icontract, 2018–2026.

[7] Hillel Wayne. TLA+ for ADTs: Refinement through named instances. Blog post, https://www.hillelwayne.com/post/tla-adt/, 2019. Pattern for composing TLA+ modules via `INSTANCE WITH` substitutions used as the basis for this pipeline's refinement obligation.

[8] Yuan Yu, Panagiotis Manolios, and Leslie Lamport. Model checking TLA+ specifications. In *Correct Hardware Design and Verification Methods (CHARME)*, volume 1703 of *LNCS*, pages 54–66. Springer, 1999.